

TOMAGI

Topological Operator Machine for Analytic Geometric Inference

Literal SDF-zero log-polar LUT, one-bit parity/jitter, kinematic change-of-change, topology, branching and deterministic query execution

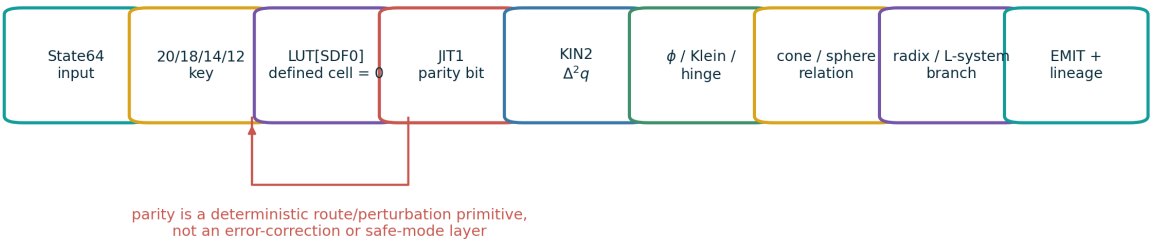
Version 1.0.0

Formal substrate definition and portable reference implementation

Requester attribution	Tom Klootwijk; NL200678942; 10-07-1990. Supplied by the requester and not independently verified.
Generated	1 September 2026.
Technical scope	43 condensed operators; 16 portable opcodes; 64-byte state; 48-byte cell; exact 64-bit key codecs; content-addressed JSON definitions; Python and C99 runtimes; GLSL, WGSL and OpenCL kernels; examples, tests, source crosswalk and reproducible validation.
Core decision	SDF-zero is literal inside the LUT. The one-bit value is a deterministic perturbation and branch. No error-correction, guard interval, damping, restoration force, confidence threshold or safe mode is injected into the runtime.

TOMAGI 1.0 literal execution chain

One stored Cell48 executes per tick; the 1-bit result selects the next cell.



Functional condensation: $\Pi(\text{Cone}(\text{LSYS}(\text{Branch}(\text{Klein}(\phi(\text{KIN2}(\text{J1T1}(\text{LUT}[\text{SDF0}](\text{K})\text{))))))))))$

Abstract

The supplied project corpus repeatedly combines a one-bit parity/jitter signal, a log-encoded polar look-up table, lower-case ϕ , second-order kinematics, an analytic cone and its circle/pyramid/sphere projections, a binary branch structure, an L-system, signed-distance-zero language and a Klein-bottle-style topology. Later source documents add content-addressed definitions, explicit operation order, a 20/18/14/12 key, portable GPU records, zero-based timeline semantics, overlap lenses, active-bit/pulse geometry and a query-first state/transition/lineage interpretation.

This report condenses those pieces into one executable definition: **TOMAGI**, the *Topological Operator Machine for Analytic Geometric Inference*. The authoritative state is a 64-byte integer record. The authoritative instruction/data object is a 48-byte LUT cell. A canonical 64-bit log-polar key selects a defined cell; defined cells are literally the SDF-zero set. A deterministic parity-of-mixed-word bit applies a bipolar perturbation and selects one of two successors. The same state then passes through integer change-of-change kinematics, lower-case- ϕ and timeline updates, reflective Klein or half-turn transport, cone/sphere relations, hinges, radix tests and a chirality-aware L-system step. Emission returns a symbolic output and a deterministic replay lineage.

The package executes the same binary program on a dependency-free Python oracle and a portable C99 evaluator. It also supplies integer-equivalent GLSL, WGSL and OpenCL kernels. The implemented claim is exact and finite: when a knowledge or control domain is compiled into literal definitions, relations, branches and output tokens, TOMAGI replaces learned prediction with deterministic operator execution.

FORMAL DECISION

TOMAGI is not an image format, neural network, probabilistic model or vendor-specific accelerator. It is a content-addressed, fixed-width state machine whose memory cell is simultaneously a definition, an operator and a zero-level relation. Projection is optional downstream output.

Contents

1	Source basis and condensation rule	1
1.1	Project sources	1
1.2	Admission and condensation	1
2	The TOMAGI program object	2
2.1	Definitions, cells and state	2
2.2	State64	2
2.3	Cell48	2
3	Log-polar coordinates and the exact 64-bit address	3
3.1	Continuous source chart and compiled integer chart	3
3.2	Contiguous key	4
3.3	Morton key	5

4	Literal SDF-zero as the memory substrate	5
5	The one-bit parity/jitter operator	6
5.1	Exact bit derivation	6
5.2	What parity does not do	7
6	Kinematic change-of-change inside the LUT	7
6.1	Integer second-order update	7
6.2	Continuous log-polar interpretation	7
7	Lower-case phi and the zero-based timeline	7
7.1	Symbol separation	7
7.2	Timeline update	7
7.3	Phi update	8
8	Topological transport	8
8.1	Reflective Klein profile	8
8.2	Source half-turn profile	8
9	Cone, circle, sphere and overlap relations	9
9.1	Finite cone source geometry	9
9.2	Packed cone relation	9
9.3	Sphere or shell relation	9
9.4	Projection semantics	9
9.5	Overlap lens	9
10	Radix routing, hinges and the L-system	9
10.1	Radix branch	9
10.2	Hinge	10
10.3	L-system transform	10
11	Transition, output and lineage	10
12	Deterministic substitution for AI	10
13	Portable instruction set and binary ABI	11
13.1	Sixteen opcodes	11
13.2	Binary file	12
14	CPU and GPU realizations	13
14.1	Python oracle	13

14.2 C99 evaluator	13
14.3 GPU kernels	13
15 Executable examples	13
15.1 Literal polar loop	14
15.2 The 19 feature pipeline	14
15.3 Compiled exact-19 rule	15
16 Validation and reproducibility	15
17 Normative no-failsafe profile	16
18 Final substrate definition	16
A Complete symbol contract	17
B Complete 43-operator catalog	18
C Exact Morton schedule	20
D Source register and crosswalk totals	21
E Package layout	22

List of Figures

1 A compiled domain replaces learned prediction with explicit deterministic execution.	2
2 The shared 64-byte state and 48-byte cell ABI.	3
3 Log-radius steps are additive in ρ and multiplicative in real radius. The origin remains an explicit core.	4
4 Canonical contiguous key and the distinct exact Morton key.	5
5 The finite LUT domain is the zero-level set. Each defined cell additionally carries an opcode and two branch successors.	6
6 The reflective radial gluing and the separately selectable source half-turn.	8
7 The binary substrate is independent of CPU or GPU implementation.	11
8 The exact reference trace and final state.	14
9 The number, binary feature set, language profile and geometry remain typed, separate objects.	15

1 Source basis and condensation rule

1.1 Project sources

Seven supplied artifacts were treated as one project corpus. Their full SHA-256 values and roles are in table 7 and in `sources/source_register.json`.

- [SRC-A] contributes the literal reordered chain in which SDF-zero is moved inside the LUT, parity is evaluated before the kinematic and structural transforms, the binary L-system routes the state, and cone/projection operators produce the output (especially pp. 16–18).
- [SRC-B] contributes the typed symbol contract, finite cone and sphere relations, log-polar chart and exact continuous kinematics, deterministic one-bit source, reflective Klein map, radix refinement and exact 20/18/14/12 key layouts (pp. 2–15).
- [SRC-C] contributes content-addressed definition nodes, dependency ordering, non-commutative hinge composition, phase-ordered evaluation, deterministic traces, active-bit filtering and pulse geometry (pp. 3–15).
- [SRC-D] contributes the zero-based offset/ordinal distinction, the one-dimensional timeline as an address sequence, the split lower-case- ϕ /double-D fold and the overlap lens as a shared domain (pp. 1–4 and 15–19).
- [SRC-E] contributes radix thresholds, $19 = 10011$, active positions $\{0, 1, 4\}$, the declared Dutch segmentation `ne|gen|tien`, a chosen three-pulse triangle and the later log-polar/Klein/ $\Delta\Delta\phi$ vocabulary (pp. 1–12 and 28–38).
- [SRC-F] contributes the mature query-first reading: a finite grammar, local spherical or radial-angular supports, directly queryable relations, explicit transitions and lineage, with projection downstream (pp. 5–12).
- [SRC-G] contributes a 211-entry normalized knowledge catalog and prior fixed-width CPU/GPU ABI material.

1.2 Admission and condensation

A source motif enters TOMAGI when it can be represented as at least one of the following: a typed coordinate, exact integer transform, finite relation, deterministic bit, topology map, branch rule, content-addressed definition, fixed-width ABI field, query operator or symbolic output. The 319-row crosswalk preserves the source identity of every admitted mechanism while mapping many source terms to a much smaller 43-operator namespace.

EVIDENCE SCOPE

The source documents include both exact mechanisms and expansive interpretations. TOMAGI does not silently assert the expansive physical conclusions. It preserves the mechanically executable content and states new engineering decisions explicitly. This evidence discipline is separate from a runtime fail-safe: it does not alter a branch, reject a state or trigger a corrective mode.

Deterministic substitution for learned inference



TOMAGI replaces a learned predictor when the domain can be compiled into finite literals, relations, branch rules and output tokens.

There are no weights, sampling temperatures, embeddings, confidence scores or hidden stochastic state in the 1.0 execution core.

Same program bytes + same input state + same tick count -> same output and replay lineage on every conforming backend.

Figure 1. A compiled domain replaces learned prediction with explicit deterministic execution.

2 The TOMAGI program object

2.1 Definitions, cells and state

A TOMAGI program is

$$P = (\mathcal{D}, C, i_0, s, m_0, q_0, F), \quad (1)$$

where $\mathcal{D}$ is a content-addressed definition registry; $C = (c_0, \dots, c_{N-1})$ is a strictly key-sorted cell table; i_0 is the entry index; s is a deterministic seed; m_0 is the default transition horizon; q_0 is the initial state; and F is the program profile word.

A literal definition node is

$$d = (id, kind, A, B, \vartheta, Dep, phase, prov, h), \quad h = \text{SHA256}(\text{canonicalJSON}(d \setminus \{h\})). \quad (2)$$

Every dependency ID must resolve and the finite definition graph must admit a topological order. This establishes one meaning for the literal program before execution.

2.2 State64

The mutable state contains sixteen 32-bit words:

$$q_n = (\rho, \theta, X, \phi, \dot{\rho}, \dot{\theta}, \dot{X}, \dot{\phi}, o, s_h, b, i, \ell, y, r, \eta). \quad (3)$$

Here o is orientation, s_h is sheet, b is the branch bit, i is the cell index, ℓ is lineage, y is output, r is the last relation residual and η is the status word. The dot notation names stored first differences; it does not imply that the integer runtime is integrating an untyped physical force.

2.3 Cell48

Each immutable cell is

$$c_i = (K_{hi}, K_{lo}, op, flags, a_0, a_1, a_2, a_3, next_0, next_1, payload, aux). \quad (4)$$

The cell is both LUT data and executable operator. The two successor indices make the one-bit branch explicit. No pointer size, heap object or vendor-specific instruction appears in the format.

Backend-neutral integer ABI

The same byte records feed Python, C99, GLSL, WGSL and OpenCL implementations.

State64 - 16 x 32-bit words

rho w0 / i32	theta w1 / i32	tick w2 / i32	phi w3 / i32	vrho w4 / i32	vtheta w5 / i32	vtick w6 / i32	vphi w7 / i32
orientation w8 / u32	sheet w9 / u32	branch w10 / u32	cell w11 / u32	lineage w12 / u32	output w13 / u32	residual w14 / i32	status w15 / u32

Cell48 - 12 x 32-bit words

key_hi w0 / u32	key_lo w1 / u32	opcode w2 / u32	flags w3 / u32	arg0 w4 / i32	arg1 w5 / i32
arg2 w6 / i32	arg3 w7 / i32	next0 w8 / u32	next1 w9 / u32	payload w10 / u32	aux w11 / u32

State64 is hot mutable state. Cell48 is immutable LUT instruction/data. Every arithmetic step is defined modulo 2^{32} , with explicit periodic normalization for theta, X and phi.

Figure 2. The shared 64-byte state and 48-byte cell ABI.

3 Log-polar coordinates and the exact 64-bit address

3.1 Continuous source chart and compiled integer chart

The source chart is

$$\rho = \ln(r/r_0), \quad \theta = \text{atan2}(y, x), \quad r = r_0 e^\rho. \quad (5)$$

Its planar metric and Jacobian are

$$ds^2 = r^2(d\rho^2 + d\theta^2), \quad J = r \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (6)$$

A compiler quantizes the declared chart into integer coordinates. The hot TOMAGI transition never calls log, exp, sin or cos.

Log-polar chart and multiplicative shells

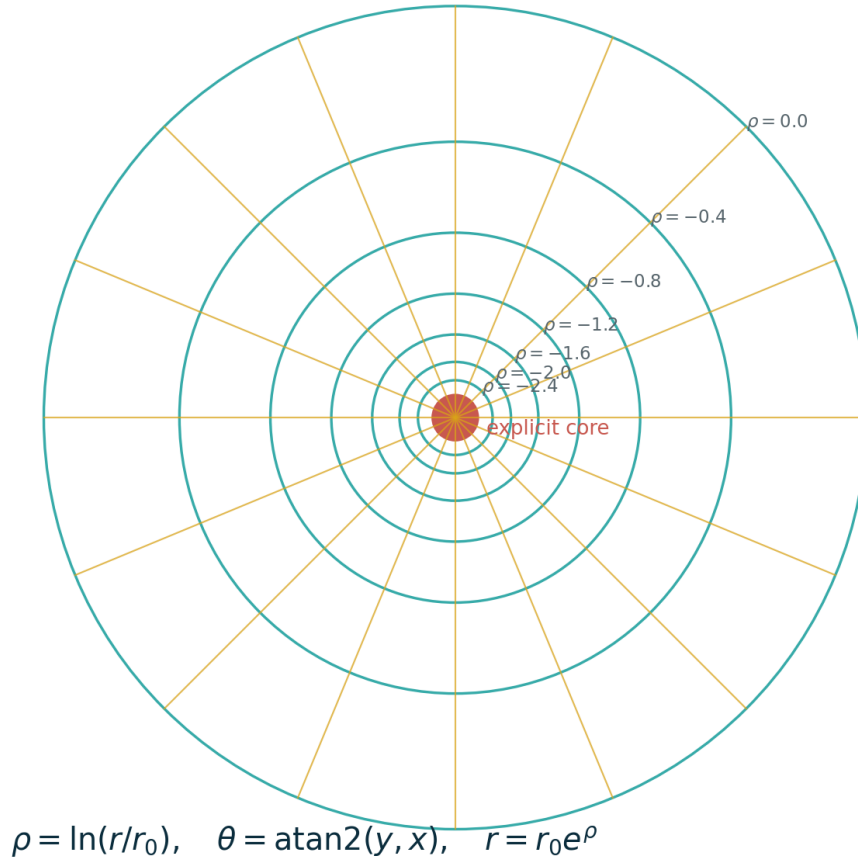


Figure 3. Log-radius steps are additive in ρ and multiplicative in real radius. The origin remains an explicit core.

3.2 Contiguous key

The canonical field widths are

Field	Bits	States	Contiguous range
q_ρ	20	2^{20}	63:44
q_θ	18	2^{18}	43:26
q_X	14	2^{14}	25:12
q_ϕ	12	2^{12}	11:0

The key is

$$K = (q_\rho \ll 44) \mid (q_\theta \ll 26) \mid (q_X \ll 12) \mid q_\phi. \quad (7)$$

The total finite address space contains exactly 2^{64} combinations.

Canonical 64-bit log-polar key

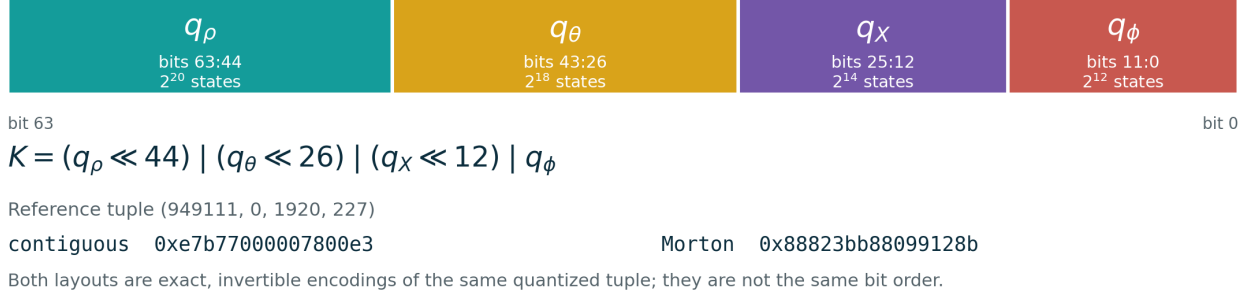


Figure 4. Canonical contiguous key and the distinct exact Morton key.

3.3 Morton key

The companion layout interleaves source bits in MSB round-robin order. It is an alternative exact codec, not an alias of the contiguous layout. For the reference tuple (949111, 0, 1920, 227):

$$K_C = 0xe7b77000007800e3, \quad (8)$$

$$K_M = 0x88823bb88099128b. \quad (9)$$

Both decode to the original tuple. The complete 64-row schedule is included as JSON, CSV and table 6.

4 Literal SDF-zero as the memory substrate

LITERAL CORE

Let D_K be the finite set of keys carried by the compiled LUT. TOMAGI defines

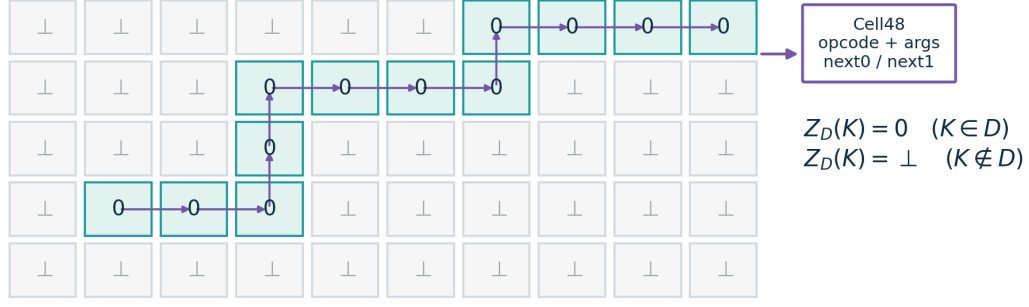
$$Z_D(K) = \begin{cases} 0, & K \in D_K, \\ \perp, & K \notin D_K. \end{cases} \quad (10)$$

Every definable cell is therefore literally on the zero level. An address outside the domain is undefined, not a positive or negative distance sample.

This definition resolves the source request to treat SDF-zero as a general operator inside the LUT rather than as an external apex-only boundary. Data storage and geometric/topological definition coincide at the cell. The `SDF0` opcode sets residual zero, marks the zero status and emits branch one. It does not contract the state toward a surface.

Literal SDF0 inside the LUT

Defined addresses are the zero-level set; non-members are undefined, not positive or negative distance.



SDF0 is not a corrective boundary in TOMAGI. It is the literal membership/value relation from which the cell's operator is evaluated.

Figure 5. The finite LUT domain is the zero-level set. Each defined cell additionally carries an opcode and two branch successors.

The complement value $\perp$ is essential. Assigning numerical distance zero to every 64-bit word would erase the distinction between a program cell and a non-address. TOMAGI instead gives the source phrase “SDF 0 on all definable points or bits” its literal finite meaning.

5 The one-bit parity/jitter operator

5.1 Exact bit derivation

Given program seed s , key words (K_h, K_l) , timeline tick X and cell salt a , define

$$h_n = \text{mix32}(s \oplus K_h \oplus \text{rotl}_{32}(K_l, 13) \oplus X \oplus a), \quad (11)$$

$$j_n = \text{popcount}(h_n) \bmod 2, \quad (12)$$

$$\sigma_n = 2j_n - 1 \in \{-1, +1\}. \quad (13)$$

The specified mixer is

```
x ^= x >> 16;
x *= 0x7feb352d;
x ^= x >> 15;
x *= 0x846ca68b;
x ^= x >> 16;
```

with all operations in $\mathbb{Z}/2^{32}\mathbb{Z}$.

For a selected state field z and cell amplitude A ,

$$z' = z + \sigma_n A, \quad b' = j_n. \quad (14)$$

Thus one bit performs two literal roles: bipolar local jitter and branch selection. Because every input to the bit is declared, repeated execution is reproducible.

5.2 What parity does not do

Parity is not interpreted as a health flag. Even and odd states are both ordinary program states. There is no hidden statement that odd means failure, no Hamming-code upgrade and no automatic return-to-boundary force. A program may route an odd branch to any cell its author declared.

6 Kinematic change-of-change inside the LUT

6.1 Integer second-order update

Collect the four coordinates and first differences as

$$p_n = (\rho_n, \theta_n, X_n, \phi_n), \quad v_n = (\dot{\rho}_n, \dot{\theta}_n, \dot{X}_n, \dot{\phi}_n). \quad (15)$$

A KIN2 cell stores

$$a_i = (a_0, a_1, a_2, a_3). \quad (16)$$

The transition is

$$v_{n+1} = \text{wrap}_{32}(v_n + a_i), \quad (17)$$

$$p_{n+1} = \text{wrap}_{32}(p_n + v_{n+1}). \quad (18)$$

This ordering is explicit and shared by all backends. The fourth acceleration component a_3 is the literal runtime $\Delta^2\phi$ operator requested in the source sequence.

6.2 Continuous log-polar interpretation

When a real-valued geometric source is compiled, SCLP supplies

$$v = r(\dot{\rho}e_r + \dot{\theta}e_\theta), \quad (19)$$

$$a = r[(\ddot{\rho} + \dot{\rho}^2 - \dot{\theta}^2)e_r + (\ddot{\theta} + 2\dot{\rho}\dot{\theta})e_\theta]. \quad (20)$$

TOMAGI treats these as host-side parameterization equations. The compiled cell contains the integer change-of-change used during execution.

7 Lower-case phi and the zero-based timeline

7.1 Symbol separation

The source corpus overloads T , time, torque and ϕ/Φ . TOMAGI imposes one contract:

- T_c denotes cone slant length in a geometric definition;
- X denotes the zero-based modular timeline tick;
- lower-case $\phi \in \mathbb{Z}/2^{12}\mathbb{Z}$ denotes phase/hoop/hinge state;
- capital $\Phi = (1 + \sqrt{5})/2$ is optional compile-time metadata and never the runtime phase.

7.2 Timeline update

For increment d_X ,

$$raw = X + d_X, \quad w = \left\lfloor \frac{raw}{2^{14}} \right\rfloor, \quad X' = raw - w2^{14}, \quad b' = w \bmod 2. \quad (21)$$

The human ordinal is $X + 1$. This is the exact interpretation of the source observation that offset 35 is the 36th position: the address remains zero based.

7.3 Phi update

The PHI opcode applies the analogous modulo- 2^{12} update. It may expose wrap parity or the half-circle bit as branch state. An odd wrap may flip orientation when the cell requests it. This makes lower-case ϕ an actual stateful hinge rather than a decorative symbol.

8 Topological transport

8.1 Reflective Klein profile

Let

$$w = \left\lfloor \frac{\rho}{2^{20}} \right\rfloor, \quad \rho' = \rho - w2^{20}. \quad (22)$$

When w is odd, the reflective profile applies

$$\theta' = 2^{17} - \theta \pmod{2^{18}}, \quad (23)$$

$$\phi' = -\phi \pmod{2^{12}}, \quad (24)$$

$$o' = o \oplus 1, \quad (25)$$

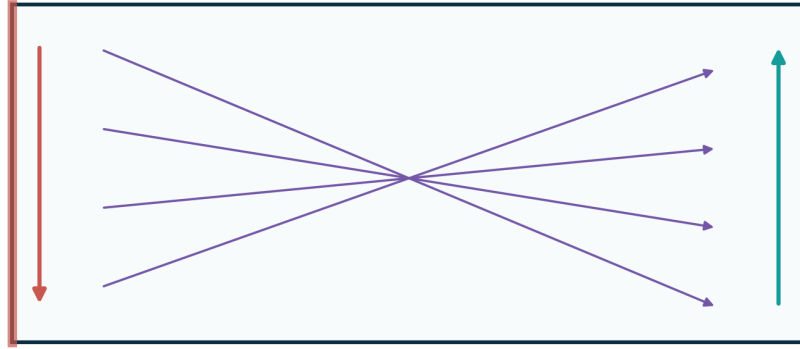
$$s'_h = s_h \oplus f_s, \quad (26)$$

$$b' = 1. \quad (27)$$

The angular reflection reverses orientation. On an even wrap, the corresponding fields remain unchanged and $b' = 0$.

Reflective Klein radial wrap

odd wrap: $\theta' = N_\theta/2 - \theta$



$$\rho' = \rho \bmod N_\rho, \quad \phi' = -\phi, \quad o' = o \oplus 1$$

Source half-turn profile remains separately selectable: $\theta' = \theta + N\theta/2$.

Figure 6. The reflective radial gluing and the separately selectable source half-turn.

8.2 Source half-turn profile

The literal source formula is preserved under a flag:

$$\theta' = \theta + 2^{17} \pmod{2^{18}}, \quad \phi' = -\phi, \quad o' = o \oplus 1. \quad (28)$$

It is a twisted state bundle but not silently called the reflective base quotient. Keeping both maps makes source fidelity and topology precise at once.

9 Cone, circle, sphere and overlap relations

9.1 Finite cone source geometry

A right circular cone is compiled from slant length $T_c > 0$ and half-angle $0 < \alpha < \pi/2$:

$$h = T_c \cos \alpha, \quad R = T_c \sin \alpha. \quad (29)$$

The exact Euclidean primitive can be evaluated by distance to its filled meridian triangle. TOMAGI's portable cell uses a quantized log-polar support relation instead of carrying floating-point primitive evaluation in every backend.

9.2 Packed cone relation

Let $\bar{\rho}$ and $\bar{\theta}$ be normalized state coordinates and δ_N the shortest cyclic difference. A **CONE** cell evaluates

$$r_C = \max\{\rho_{min} - \bar{\rho}, \bar{\rho} - \rho_{max}, |\delta_{2^{18}}(\bar{\theta}, \theta_0)| - |\alpha_q|\}. \quad (30)$$

It sets $b = 1$ precisely when $r_C \leq 0$.

9.3 Sphere or shell relation

A **SPHERE** cell evaluates

$$r_S = \max\{|\bar{\rho} - \rho_0| - |d_\rho|, |\delta_{2^{12}}(\bar{\phi}, \phi_0)| - |d_\phi|\}. \quad (31)$$

A negative d_ϕ disables the phase component and yields a pure radial shell.

9.4 Projection semantics

The side-view pyramid, circle and sphere are symbolic projection tokens. A cone viewed through the triangular profile may emit **PYRA**; its axial radial profile may emit **CIRC**; a spherical support may emit a sphere token. The state and relation remain authoritative whether or not any pixels are later drawn.

9.5 Overlap lens

The source's slight overlap is the exact shared domain

$$L = A \cap B. \quad (32)$$

It can be compiled as conjunction, join eligibility or a connector hinge. TOMAGI does not assign an unstated physical energy meaning to the lens.

10 Radix routing, hinges and the L-system

10.1 Radix branch

RADIX(k) selects bit k from the canonical key. Repeated tests create a radix-2 trie over the packed state. The operation refines an address prefix; it does not divide the decoded coordinates by two.

10.2 Hinge

The compiled hinge is

$$H = (\text{state}, M_0, M_1, \text{invariant metadata}). \quad (33)$$

M_0 is identity. If $b = 1$, M_1 adds the cell's four arguments to (ρ, θ, X, ϕ) and optionally flips orientation and sheet. Hinges compose in stored order and are not assumed to commute.

This interface covers geometric rotation after quantization, parity-selected routing, a zero-magnitude connector such as Dutch *en*, and the split- ϕ /double-D fold when the definition explicitly declares those semantics.

10.3 L-system transform

For turn quantum τ and dyadic shift d ,

$$\chi = (-1)^o, \quad (34)$$

$$s_b = 2b - 1, \quad (35)$$

$$\phi' = \phi + \chi s_b \tau \pmod{2^{12}}, \quad (36)$$

$$v' = \text{trunc}_0(v/2^d). \quad (37)$$

This is the one-step executable condensation of

$$F(T) \Rightarrow F(T/2)[\pm \phi F(T/2)]. \quad (38)$$

The finite program graph, not a hidden recursion manager, determines the actual sequence of steps.

11 Transition, output and lineage

After an opcode, periodic fields are normalized and lineage is updated:

$$\ell' = \text{mix32}(\ell \oplus \text{payload} \oplus \text{aux} \oplus K_h \oplus \text{rotl}_{32}(K_l, 7) \oplus b \oplus i). \quad (39)$$

The next cell is

$$i' = \begin{cases} \text{next}_0, & b = 0, \\ \text{next}_1, & b = 1. \end{cases} \quad (40)$$

A flagged cell may instead re-key the normalized state and binary-search the sorted LUT. A miss is observable in status and falls through to the declared successor; no replacement state is invented.

PROJECT writes a token and continues. EMIT writes a token and marks an output; its low flag bit may terminate. HALT terminates without changing the current token. A trace records the exact operator order and every resulting state field.

12 Deterministic substitution for AI

FORMAL DEFINITION

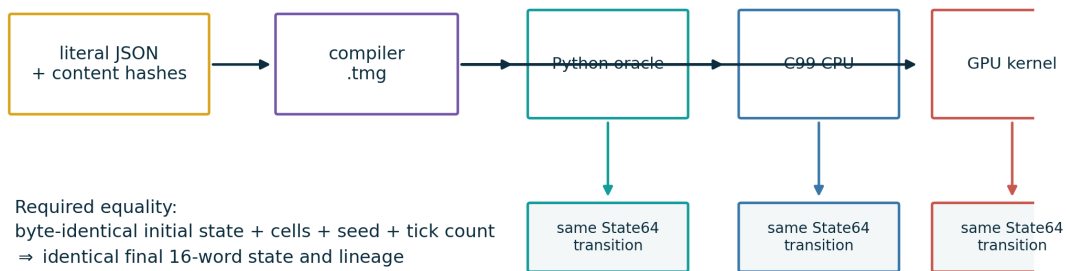
A task is TOMAGI-compilable when its input has a finite typed encoding; its facts and transformations can be expressed as finite definitions, relations or branch rules; and its answer has a finite output representation. Its deterministic inference function is

$$\text{answer} = \text{Exec}(\text{Compile}(D, R), \text{Encode}(x), m).output. \quad (41)$$

There are no learned weights, embeddings, sampling temperatures, confidence scores or hidden random state in this execution. The one-bit sequence is a pure function of explicit inputs. This makes TOMAGI suitable for compiled classification, finite-state planning, procedural generation, relation queries, spatial routing, signal-state transforms and symbolic rule systems.

The distinction from a learned model is exact. A learned model may statistically generalize beyond its training examples. TOMAGI evaluates what has been made literal. New knowledge enters by adding or compiling definitions and cells, not by changing opaque weights.

One binary program, multiple deterministic backends



The supplied validation executes Python and C against the same .tmg file. GLSL, WGSL and OpenCL sources implement the same ABI and opcode equations.

Figure 7. The binary substrate is independent of CPU or GPU implementation.

13 Portable instruction set and binary ABI

13.1 Sixteen opcodes

The 16 hot opcodes are shown in table 1. Meta-level operators such as definition hashing, compilation, overlap conjunction and trace explanation remain in the 43-entry catalog but do not consume opcode numbers.

Table 1. The sixteen portable Cell48 opcodes

Code	Name	State effect
0	NOP	No state mutation beyond lineage and successor selection.
1	SET	Set one State64 field selected by flags[3:0] to arg0.
2	JIT1	Derive a deterministic parity bit and add +/-arg0 to the selected field.
3	KIN2	Apply integer second-order update $v'=v+args$; $q'=q+v'$.
4	PHI	Advance lower-case phi modulo 2^{12} and expose wrap/half-circle branch.
5	TIME	Advance zero-based modular tick modulo 2^{14} and fold winding into lineage.
6	SDF0	Return literal zero relation for the current defined LUT cell.
7	CONE	Evaluate a quantized log-polar cone/support relation.
8	SPHERE	Evaluate a quantized radial/phi shell relation.
9	KLEIN	Apply reflective Klein or source half-turn radial wrap.
10	RADIX	Select one bit of the canonical 64-bit key.
11	HINGE	Apply map1 deltas and optional orientation/sheet flips when branch=1.
12	LSYS	Apply chirality-signed lower-case-phi turn and dyadic velocity scaling.

continued on next page

The sixteen portable Cell48 opcodes – continued

Code	Name	State effect
13	PROJECT	Set a symbolic downstream output token without halting.
14	EMIT	Set output token and emitted status; flag bit 0 may terminate the program.
15	HALT	Terminate the current state execution.

13.2 Binary file

A `.tmg` program begins with a 128-byte little-endian header:

Byte	Size	Field
0	8	ASCII TOMAGI1\0
8	4	version 0x00010000
12	4	program flags
16	4	cell count
20	4	entry index
24	4	seed
28	4	default ticks
32	4	cell size, 48
36	4	state size, 64
40	24	reserved zero words
64	64	initial State64

The sorted cell table starts at byte 128. Total size is $128 + 48N$ bytes.

Table 2. State64 ABI

Word	Byte	Field	Type	Meaning
0	0	rho	i32	log-radius coordinate; canonical key uses modulo 2^{20}
1	4	theta	i32	polar angle; modulo 2^{18}
2	8	tick	i32	zero-based timeline tick; modulo 2^{14}
3	12	phi	i32	lower-case phase/hinge angle; modulo 2^{12}
4	16	vrho	i32	first difference of rho
5	20	vtheta	i32	first difference of theta
6	24	vtick	i32	first difference of tick
7	28	vphi	i32	first difference of phi
8	32	orientation	u32	orientation/chirality bit
9	36	sheet	u32	sheet index
10	40	branch	u32	current branch bit
11	44	cell	u32	current cell index
12	48	lineage	u32	deterministic replay checksum
13	52	output	u32	symbolic output token
14	56	residual	i32	last signed relation residual
15	60	status	u32	status bit field

Table 3. Cell48 ABI

Word	Byte	Field	Type	Meaning
0	0	key_hi	u32	canonical key bits 63..32
1	4	key_lo	u32	canonical key bits 31..0
2	8	opcode	u32	opcode 0..15

continued on next page

Cell48 ABI – continued

Word	Byte	Field	Type	Meaning
3	12	flags	u32	generic and opcode-specific flags
4	16	arg0	i32	operator argument 0
5	20	arg1	i32	operator argument 1
6	24	arg2	i32	operator argument 2
7	28	arg3	i32	operator argument 3
8	32	next0	u32	successor for branch 0
9	36	next1	u32	successor for branch 1
10	40	payload	u32	literal/output/provenance token
11	44	aux	u32	literal/hash salt/operator auxiliary

14 CPU and GPU realizations

14.1 Python oracle

The dependency-free Python package supplies canonical JSON, definition hash verification, topological dependency ordering, the two key codecs, binary serialization, the exact step function and a JSON trace CLI. It is the readable reference semantics.

14.2 C99 evaluator

The C implementation reads little-endian bytes explicitly and does not rely on compiler structure packing. It defines floor division, two's-complement conversion, modular addition and cyclic differences directly. The same `.tmg` file is therefore portable across ordinary CPU architectures.

14.3 GPU kernels

The GLSL 4.50, WGSL and OpenCL C kernels use the same three logical inputs:

1. read-write `State64[]` array;
2. read-only sorted `Cell48[]` array;
3. parameters (state count, cell count, seed, reserved).

One invocation executes one transition for one independent state. Repeated dispatches advance the query horizon. The kernels require ordinary 32-bit integer storage, shifts, addition, comparison and bit count. They make no vendor, tensor-core, ray-tracing-core or floating-point performance assumption.

EVIDENCE SCOPE

This package executed Python and C99 and verified exact equality. The OpenCL source was accepted by Clang's OpenCL 1.2 parser. The build environment did not contain a GLSL/WGSL validator or an accessible physical GPU dispatch runtime, so those two sources are supplied against the shared ABI but are not claimed as device-executed in this validation run.

15 Executable examples

15.1 Literal polar loop

The primary example executes

$$\text{SDF0} \rightarrow \text{JIT1} \rightarrow \text{KIN2} \rightarrow \text{KLEIN} \rightarrow \text{HINGE} \rightarrow \text{LSYS} \rightarrow \text{CONE} \rightarrow \text{EMIT}. \quad (42)$$

Starting near the radial wrap, the kinematic step crosses the 2^{20} boundary. The reflective Klein cell returns ρ to 8, flips orientation and sheet, and routes to the hinge. The final cone relation emits `0x50595241`, the ASCII token PYRA.

Executable polar-loop trace

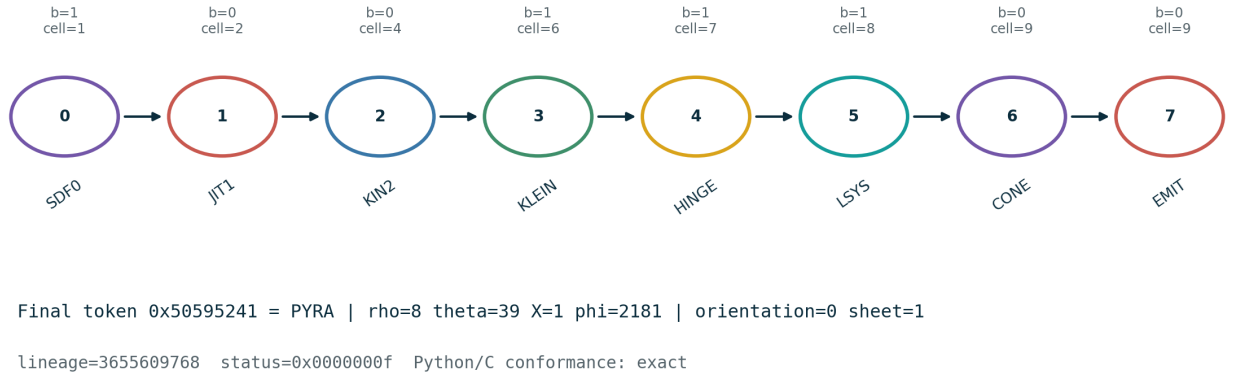


Figure 8. The exact reference trace and final state.

15.2 The 19 feature pipeline

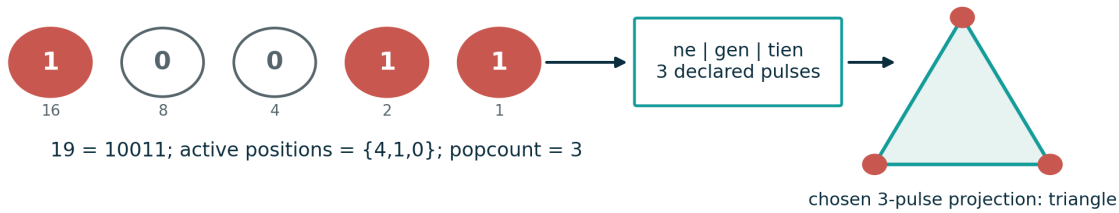
The source-derived example keeps three representations distinct:

$$19 = 10011_2, \quad A_2(19) = \{0, 1, 4\}, \quad |A_2(19)| = 3, \quad (43)$$

$$\Sigma_{nl}(19) = (\text{ne}, \text{gen}, \text{tien}), \quad |\Sigma_{nl}(19)| = 3. \quad (44)$$

The equality of the two counts produces a metadata bit. Three declared pulses are projected to a triangle. The decimal value remains 19 and the binary word remains 10011.

Source-derived 19 feature pipeline



Typed result

numeric value remains 19; binary word remains 10011; Dutch segmentation and triangle are separate, linked representations.

The feature-count equality is executable metadata, not an arithmetic rewrite.

Figure 9. The number, binary feature set, language profile and geometry remain typed, separate objects.

15.3 Compiled exact-19 rule

The `exact19_rule.json` example encodes its input in ρ and uses a zero-width cone relation with full angular support:

$$r = \max(19 - \rho, \rho - 19, |\delta(\theta, 0)| - 2^{17}). \quad (45)$$

It emits token 19 for $\rho = 19$ and token 0 for $\rho = 18$. This small program illustrates the intended AI-substitution mechanism: an explicit relation is compiled to a branch and finite output, with a complete trace.

16 Validation and reproducibility

The package's `make validate` workflow regenerates specifications and examples, builds the C99 evaluator, executes the Python tests, compares Python and C states, syntax-checks OpenCL when Clang is available, regenerates figures and writes a machine-readable validation report.

Check	Result
Python conformance suite	24 tests passed
Python/C polar-loop state	exact match, all 16 fields
Python/C exact-19 state	exact match, all 16 fields
Contiguous reference key	0xe7b77000007800e3
Morton reference key	0x88823bb88099128b
JSON examples	valid under Draft 2020-12 schema
OpenCL C kernel	Clang syntax check passed
GLSL and WGSL	structural source checks passed; no external compiler/device in environment
Source condensation	43 operators, 319 crosswalk rows, 7 source artifacts

The canonical reproduction commands are:

```
make validate
make report

PYTHONPATH=src/python python -m tomagi compile \
  examples/polar_loop.json examples/polar_loop.tmg
PYTHONPATH=src/python python -m tomagi run \
  examples/polar_loop.tmg --trace
./build/tomagi-c examples/polar_loop.tmg
```

17 Normative no-failsafe profile

NORMATIVE NO-FAILSAFE PROFILE

TOMAGI 1.0 does not include the following earlier-draft layers in its transition algebra:

- jitter interval/event-margin guard
- boundary restoration/contraction force
- confidence threshold
- dynamic allocation safety ring
- Hamming or ECC correction
- damping or emergency retreat
- safe mode or watchdog route
- claims-ledger kill criterion

Parity is an ordinary branch. SDF0 returns zero at every defined cell. No hidden policy changes those results.

A finite file still has a grammar: record sizes, version, unique keys, resolvable references and in-range successors define whether bytes can be parsed as a TOMAGI program. These are syntax and identity rules, not a runtime correction mechanism.

18 Final substrate definition

FORMAL DEFINITION

TOMAGI 1.0 is a finite, content-addressed, deterministic operator machine in which a 20/18/14/12 log-polar key selects a literal SDF-zero LUT cell; a parity-of-mixed-word one-bit operator produces signed jitter and a branch; integer second-order kinematics advances log-radius, angle, zero-based time and lower-case phi; reflective Klein or half-turn maps transport orientation and sheet; cone, sphere, overlap, radix, hinge and L-system operators transform or classify the state; branch-indexed successors serialize non-commutative composition; and emission plus lineage returns a replayable symbolic answer. The same 64-byte state and 48-byte cell definitions execute on the supplied Python and C CPU runtimes and map directly to GLSL, WGSL and OpenCL GPU kernels.

For compiled domains, this is the deterministic substitute for AI: definitions replace weights, relation cells replace opaque activations, the one-bit route replaces stochastic sampling, and the trace exposes every transition that produced the answer.

A Complete symbol contract

Table 4. TOMAGI 1.0 symbol contract

Symbol	Typed meaning	Representation
D	finite domain of definable LUT keys	set of 64-bit keys
d	content-addressed definition node	typed record
q_n	TOMAGI state at tick n	State64
rho	log-radius coordinate	20-bit periodic integer; decoded range profile [-20,0]
theta	polar angle	18-bit periodic integer
X	zero-based modular timeline tick	14-bit periodic integer
phi	lower-case phase/hoop/hinge coordinate	12-bit periodic integer
Phi	capital golden ratio	optional constant; never aliases runtime phi
Delta q	first discrete difference	velocity
Delta ² q	second discrete difference	acceleration stored in a cell
Delta ² phi	change-of-change of lower-case phi	cell arg3 under KIN2
K=(K_hi,K_lo)	contiguous 64-bit log-polar key	two u32 words
LUT_Z	finite sorted table of SDF0 cells	Cell48[]
Z_D	literal SDF0 zero-field	0 on D, bottom outside D
P	one-bit parity reduction	popcount mod 2
j	deterministic parity/jitter bit	0 or 1
sigma	signed jitter selector	2j-1 in {-1,+1}
T_c	cone slant length	real-valued compile-time geometry parameter
alpha	cone half-angle	compile-time or quantized angular width
C	cone relation/support	signed relation
S	sphere/shell relation/support	signed relation
W_K	Klein or half-turn wrap map	state transform
b	branch bit	0 or 1
i	compiled cell index	u32
ell	lineage checksum	u32
y	output token	u32
Pi	projection/tag operator	symbolic downstream output
bottom	undefined/non-member	not a numeric SDF value

B Complete 43-operator catalog

The catalog includes both the 16 hot opcodes and meta/compiler operators. The machine-readable JSON and CSV add source basis and canonical content hashes.

Table 5. TOMAGI 1.0 complete 43-operator catalog

ID	Operator	Signature	Literal definition
TOM.DEF	definition node	record -> address	A typed in-substrate definition $d=(id,kind,A,B,parameters,dependencies,phase,provenance,hash)$.
TOM.REF	reference resolution	definition graph -> ordered graph	Resolve every definition_ref and dependency ID; order dependencies before dependents.
TOM.SEQ	ordered composition	operators ⁿ -> operator	Composition is serialized and non-commutative; execute the stored order without re-sorting.
TOM.INDEX0	zero-based position	offset -> ordinal	ordinal = offset + 1; the stored timeline coordinate remains the zero-based offset.
TOM.LOGPOLAR	log-polar chart	(x,y,r0) -> (rho,theta)	$\rho=\ln(r/r_0)$, $\theta=\text{atan2}(y,x)$; runtime consumes quantized ρ/θ already compiled.
TOM.KEY.C	contiguous 64-bit key	$\rho_{20} \times \theta_{18} \times X_{14} \times \phi_{12} \rightarrow K$	$K=(\rho \ll 44) (\theta \ll 26) (X \ll 12) \phi$.
TOM.KEY.M	Morton 64-bit key	$\rho_{20} \times \theta_{18} \times X_{14} \times \phi_{12} \rightarrow KM$	Interleave MSB round-robin: $\rho_{19}, \theta_{17}, X_{13}, \phi_{11}, \dots$ until all 64 bits are consumed.
TOM.FETCH	LUT cell lookup	$K \rightarrow \text{Cell or undefined}$	Resolve a canonical key in the sorted cell table; compiled graph mode may use direct successor indices.
TOM.SDF0	literal zero field	defined key -> 0	$Z_D(K)=0$ for every K in the finite domain D of definable cells; $Z_D(K)=\text{bottom}$ otherwise.
TOM.PARITY	one-bit reduction	$u_{32} \rightarrow \text{bit}$	$P(w)=\text{popcount}(w) \bmod 2$.
TOM.JIT1	one-bit jitter	seed $\times K \times X \rightarrow \{-a,+a\}$	$j=P(\text{mix32}(\text{seed} \text{ xor } K_hi \text{ xor } \text{rotl}(K_lo,13) \text{ xor } X \text{ xor } aux));$ $\sigma=2^j-1$; selected state field += $\sigma \times a$.
TOM.DELTA1	first discrete difference	$q_n, q_n(n-1) \rightarrow v_n$	$\Delta q_n = q_n - q_n(n-1)$.
TOM.DELTA2	second discrete difference	$v_n, v_n(n-1) \rightarrow a_n$	$\Delta^2 q_n = v_n - v_n(n-1)$; $\Delta^2 \phi$ is the angular acceleration coordinate.
TOM.KIN2	second-order update	state $\times$ acceleration-cell $\rightarrow$ state	$v_n(n+1)=v_n+a_n$; $q_n(n+1)=q_n+v_n(n+1)$.
TOM.PHI	lower-case phi hinge	$\phi \times \Delta \rightarrow \phi$	ϕ is periodic in $\mathbb{Z}/(2^{12}\mathbb{Z})$; capital Φ is reserved for the golden ratio and is not the runtime phase.
TOM.TIME	timeline phase	$X \times \Delta \rightarrow X, \text{winding}$	X is a 14-bit modular tick; wraps contribute to lineage while the tick remains periodic.
TOM.CONE	analytic cone relation	packed chart $\times$ cone parameters $\rightarrow$ signed relation	$r=\max(\rho_min-\rho, \rho-\rho_max, \text{cyclic}(\theta-\theta_0) -\alpha_q)$; inside iff $r \leq 0$.
TOM.SWEEP	swept cone family	cone $\times$ path $\rightarrow$ relation	$F_sweep(x)=\inf_u d_C(x-s(u))$; a compiled profile may tabulate the path as KIN2 cells.
TOM.CIRCLE	circle/radial shell	$\rho \times \text{center}, \text{width} \rightarrow$ signed relation	A circle is a radial shell in the packed chart and an axial projection of a cone.
TOM.SPHERE	spherical support	$\rho, \phi \times \text{center}, \text{width} \rightarrow$ signed relation	Packed shell residual is $\max(\rho-\rho_0 -\text{dr}, \text{cyclic}(\phi-\phi_0) -\text{d}\phi)$; Euclidean helper is $\ x-c\ -R$.
TOM.OVERLAP	lens intersection	$A, B \rightarrow A \text{ intersect } B$	Shared domain is exact conjunction/intersection; it may be used as a join key or connector locus.
TOM.KLEIN	reflective Klein wrap	state $\rightarrow$ state	On an odd radial wrap: $\rho'=\rho \bmod N_rho$, $\theta'=N_theta/2-\theta$, $\phi'=-\phi$, $\text{orientation}'=\text{orientation} \text{ xor } 1$; optional sheet flip.
TOM.MOBIUS	half-turn bundle	state $\rightarrow$ state	On an odd wrap: $\theta'=\theta+N_theta/2$, $\phi'=-\phi$, $\text{orientation}'=\text{orientation} \text{ xor } 1$.
TOM.RADIX	radix branch	$K \times \text{bit-index} \rightarrow \text{bit}$	Select one canonical key bit; repeated selections form a radix-trie path.
TOM.HINGE	typed hinge	branch $\times$ state $\times$ map $\rightarrow$ state	$H=(\text{pivot}/\text{state}, \text{map0}, \text{map1}, \text{invariant})$; runtime map0 is identity and map1 is the cell's declared Δ plus optional orientation/sheet flip.
TOM.LSYS	binary L-system step	branch $\times$ state $\rightarrow$ state	Apply chirality-signed turn and dyadic velocity scaling, corresponding to $F(T) \rightarrow F(T/2)[+/-\phi]$ $F(T/2)$.
TOM.BRANCH	binary successor	bit $\times$ cell $\rightarrow$ cell	$i_n(n+1)=\text{next_i}[bit]$.
TOM.TRANSITION	state patch	state $\times$ cell $\rightarrow$ state	Execute exactly one opcode, normalize periodic fields, select successor, and update lineage.

continued on next page

TOMAGI 1.0 complete 43-operator catalog – continued

ID	Operator	Signature	Literal definition
TOM.PROJECT	symbolic projection	state x tag -> tag	Emit a side-view pyramid, circle, sphere, action, class, or other downstream token without making projection the substrate.
TOM.EMIT	deterministic output	state x payload -> state	Set the output token and emitted bit; optional halt is a program semantic, not a safety mode.
TOM.LINEAGE	replay lineage	state x cell x key -> u32	ell'=mix32(ell xor payload xor aux xor K_hi xor rotl(K_lo,7) xor branch xor cell_index).
TOM.TRACE	explanation trace	execution -> records	Return ordered cell IDs/opcodes, branches, keys, residuals, outputs and lineage.
TOM.ACTIVE	active-bit projection	integer -> set(bit positions)	A(n)={k bit_k(n)=1}; the original numeric value is retained.
TOM.POPCOUNT	active-bit cardinality	bit set -> integer	Return the number of active positions in a finite binary word.
TOM.EQ	typed equality predicate	A x A -> bit	Return 1 exactly when two values of the same declared type are equal; the result is a branch bit, not an identity rewrite.
TOM.PASCAL2	Pascal parity	n,k -> bit	C(n,k) mod 2 defines a deterministic triangular occupancy pattern.
TOM.PULSE	pulse geometry	count -> point/segment/m-gon	m declared pulses map to one point, a segment, a triangle for m=3, or a regular m-gon.
TOM.RULE	compiled rule	facts x predicates -> cell graph	Translate exact predicates into RADIX/CONE/SPHERE/HINGE branches and EMIT leaves.
TOM.QUERY	deterministic inference	program x state x ticks -> output,trace	Run the finite operator graph for the requested number of ticks or until an EMIT/HALT cell.
TOM.ABI.STATE64	hot state record	16 words -> state	64-byte state: four coordinates, four velocities, orientation/sheet/branch/cell, lineage/output/residual/status.
TOM.ABI.CELL48	LUT cell record	12 words -> instruction cell	48-byte cell: key, opcode/flags, four signed args, two successors, payload, aux.
TOM.BACKEND.CPU	portable CPU evaluator	TMG program -> state	Dependency-free Python oracle and C99 evaluator implement identical integer semantics.
TOM.BACKEND.GPU	portable GPU step	state[] x cell[] -> state[]	One invocation executes one TOMAGI transition using only 32-bit integer storage and bit operations.

C Exact Morton schedule

Table 6. Exact 64-row MSB round-robin Morton schedule

Output bits 63–32			Output bits 31–0		
Out	Field	Src	Out	Field	Src
63	rho	19	31	rho	11
62	theta	17	30	theta	9
61	tick	13	29	tick	5
60	phi	11	28	phi	3
59	rho	18	27	rho	10
58	theta	16	26	theta	8
57	tick	12	25	tick	4
56	phi	10	24	phi	2
55	rho	17	23	rho	9
54	theta	15	22	theta	7
53	tick	11	21	tick	3
52	phi	9	20	phi	1
51	rho	16	19	rho	8
50	theta	14	18	theta	6
49	tick	10	17	tick	2
48	phi	8	16	phi	0
47	rho	15	15	rho	7
46	theta	13	14	theta	5
45	tick	9	13	tick	1
44	phi	7	12	rho	6
43	rho	14	11	theta	4
42	theta	12	10	tick	0
41	tick	8	9	rho	5
40	phi	6	8	theta	3
39	rho	13	7	rho	4
38	theta	11	6	theta	2
37	tick	7	5	rho	3
36	phi	5	4	theta	1
35	rho	12	3	rho	2
34	theta	10	2	theta	0
33	tick	6	1	rho	1
32	phi	4	0	rho	0

D Source register and crosswalk totals

Table 7. Project source register

ID	Source	SHA-256	Role
SRC-A	1bit parity / lower-case phi / log-polar LUT / cone / Klein / L-system dialogue	ede0fd37504b07fa...f9833368	Literal SDF0-in-LUT chain and precedence.
SRC-B	UGTS-KC 3.6.2 SCLP	6987490a0a7b7ed7...3f5ea580	Cone, log-polar kinematics, jitter, phi, Klein wrap, L-system, 64-bit keys.
SRC-C	UGTS-KC 3.6 literal referential substrate	e21433cd8378368d...d6003a46	Definition records, hinges, operation order, active-bit and pulse examples.
SRC-D	21 Ben Burgers Strikes Back	1ab4f04669ded1aa...8ef0f6dd	Zero-based timeline, split-phi fold, overlap lens and connector interpretation.
SRC-E	19 / binary threshold / Dutch phonetic dialogue	6e6150026e728a68...e7db58de	Active bits, pulse triangle, log-polar and Klein motifs.
SRC-F	Chronological Synthesis of the Spherical Substrate Line	9bacfe071a64951f...6036e052	Query-first relation, transition and lineage synthesis.
SRC-G	UGTS spatial distill and GSP4 package	e2149593ef18c3b5...e6501313	211-mechanism catalog, CPU oracle, GPU ABI/shader sources and deterministic event authority.

Crosswalk row count by source

Source	Crosswalk rows
SRC-A	6
SRC-B	50
SRC-C	36
SRC-D	4
SRC-E	6
SRC-F	6
SRC-G	211
Total	319

The complete 319-row table is supplied in `spec/source_crosswalk.json` and `spec/source_crosswalk.csv` rather than expanded across the report.

E Package layout

```
TOMAGI_1_0_Tom_Klootwijk/  
  README.md  
  CHANGELOG.md  
  NOTICE.md  
  VERSION  
  Makefile  
  pyproject.toml  
  report/  
    TOMAGI_1_0_Tom_Klootwijk.pdf  
    TOMAGI_1_0_Tom_Klootwijk.tex  
    figures/*.png  
    generated/*.tex  
  spec/  
    TOMAGI_1_0_FORMAL_DEFINITION.md  
    tomagi.schema.json  
    operator_catalog.{json,csv}  
    symbol_table.{json,csv}  
    opcode_table.{json,csv}  
    state64_layout.{json,csv}  
    cell48_layout.{json,csv}  
    status_flags.{json,csv}  
    key_layout.json  
    morton_schedule_64.{json,csv}  
    precedence.json  
    source_crosswalk.{json,csv}  
    BINARY_FORMAT.md  
    GPU_ABI.md  
    NO_FAILSAFE_CORE.md  
  src/python/tomagi/*.py  
  src/c/tomagi.{h,c}  
  src/c/tomagi_cli.c  
  src/gpu/tomagi_step.{comp,wgsl,cl}  
  examples/*.json  
  examples/*.tmg  
  tests/test_tomagi.py  
  validation/*  
  sources/*  
  checksums/SHA256SUMS.txt
```